Thetawave.ai

💻 工廠方法模式 – 設計條件與實作解析

Generated by Thetawave.ai

🏭 工廠方法模式（Factory Method）介紹與需求背景

> **工廠方法**是一種創建型設計模式，用來解決在不確定實際要創建哪種產品類別時，如何延遲決定並在適當時機創建物件的問題。它是樣板方法（Template Method）的一種特殊形式，專注於將「變動的創建行為」與「不變的行為」區分開來。

開發背景：聊天伺服器框架（Chat Server Framework）

- 目標是建立一個簡易聊天伺服器框架，負責網路基礎建設並提供基本日誌功能。
- 啟動伺服器時會先啟動一個 TCP Socket Server（TCP 網路協定與 Socket 是網路底層細節，學生不必熟悉）。
- 客戶端（Client）透過連接 Socket Server 建立連線，伺服器為每個 Client 創建一個傳聲筒（Tunnel），雙向通訊。
- **通訊協定（Protocol）**：雙方必須遵守的訊息傳遞規則，例如必須先喊「水球好棒」才能傳訊息，或者訊息需加密後才傳送。
- 要求框架能讓開發者彈性客製化不同的通訊協定，因此同事已定義好一個 **Tunnel 介面**，負責在特定協定下監聽 Client 訊息並在訊息讀取後或違規時斷開連線。

🧩 物件導向分析與設計難題

物件類別分析

類別名稱	角色與責任

Chat Server	負責啟動 TCP Socket Server，管理 Client 連線，依賴 Tunnel 介面與 Client 通訊
Tunnel (介面)	傳聲筒，負責監聽 Client 訊息、傳遞訊息及斷開連線
Socket Server	TCP 網路入口，接收 Client 連線
Socket Client	代表連接的 Client 對象

主要流程

1. 啟動 Chat Server，啟動 TCP Socket Server。
2. TCP Server 接收新 Client 連線（accept）。
3. 為每個 Client 創建一個 Tunnel 物件，透過該 Tunnel 監聽並接收訊息。
4. Tunnel 讀取 Client 訊息後，立即斷開該 Client 連線。

設計難點（Forces）

Force 編號	說明
Force 1	**Tunnel 介面物件只能使用一次**：Tunnel 的 Message 方法使用後即斷線，Tunnel 物件即報廢，不能重複使用。
Force 2	**遵守開放封閉原則（Open-Close Principle，OCP）**：希望未來新增不同的 Tunnel 實作時，不必修改 Chat Server 程式碼。

兩個 Force 互相衝突：

- 由於 Tunnel 物件一次性，必須在每次 Client 連線時創建新的 Tunnel 實作。
- 但同時又不知未來會有多少種 Tunnel 實作類別，需要高度彈性。

設計問題（Problem）：如何在不知實際要創建哪種 Tunnel 實作類別的情況下，於適當時機為每一個 Client 創建對應的 Tunnel 物件？

⚙️ 工廠方法模式的解決方案

Thetawave.ai

變與不變的區分

- **不變的部分**：Chat Server 整體啟動伺服器、管理 Client 連線的流程。
- **變動的部分**：創建 Tunnel 物件的行為，因為不同專案或需求會有不同的通訊協定實作。

工廠方法模式核心

- Chat Server 類別定義一個抽象的工廠方法（Factory Method）`CreateTunnel(Client)`，用來創建對應 Client 的 Tunnel。
- 具體的子類別（Concrete Context）覆寫此工廠方法，決定要創建哪一種 Tunnel 實作。
- Chat Server 啟動時，呼叫此工廠方法，取得具體 Tunnel 物件，完成通訊。

工廠方法模式的類別關係示意

類別	職責說明
Chat Server (抽象)	定義啟動流程與抽象的 `CreateTunnel` 工廠方法
Concrete Chat Server	繼承 Chat Server，覆寫 `CreateTunnel`，指定 Tunnel 類別
Tunnel (介面)	定義訊息接收和斷開連線的介面
Concrete Tunnel	實作 Tunnel 介面，依協定處理訊息

🔄 依賴注入與為何不可直接注入 Tunnel

- **一開始嘗試依賴注入失敗原因**：
  - Tunnel 是一次性物件，且必須知道服務的 Client。
  - 如果從外部注入 Tunnel，無法在 Chat Server 中依照每個 Client 動態創建新的 Tunnel 實例。
- **解決方法**：
  - 將 Tunnel 的創建行為抽象成工廠方法，留給子類別決定創建哪個 Tunnel。

- 或改用**組合取代繼承**，將工廠方法委派給一個獨立的 Tunnel Factory 介面，再由 Chat Server 組合使用。

metawave.ai

🧱 組合取代繼承的優化設計

問題：組合爆炸（Combinatorial Explosion）

- 假如 Chat Server 不只一個可變行為（例如，除了創建 Tunnel，還要創建 Database），
  若使用繼承覆寫多個工廠方法，會導致需要實作大量子類別，組合數量呈指數成長。

解決方案：將可變行為委派出去（依賴反轉）

設計方式	特點說明
繼承覆寫工廠方法	子類別需覆寫多個工廠方法，容易導致類別爆炸
委派給工廠介面（Tunnel Factory）	透過組合把創建行為委派給獨立介面，避免類別爆炸，增強彈性

重構後的架構

- 定義一個 TunnelFactory 介面，包含 CreateTunnel(Client) 方法。
- Chat Server 擁有一個 TunnelFactory 實例，啟動時用此工廠建立 Tunnel。
- 開發者可自行實作不同的 TunnelFactory，並透過依賴注入組合進 Chat Server。

💡 實作範例與流程概述

階段	動作說明

啟動 Chat Server	呼叫 StartServer()，啟動 TCP Socket Server 監聽 Port
接受 Client 連線	TCP Server 接收到 Client 連線，回傳 Client 對應 Socket 物件
創建 Tunnel	呼叫 CreateTunnel(Client)（工廠方法或由 TunnelFactory 實作），取得 Tunnel 實例
監聽訊息與斷線	Tunnel 物件監聽 Client 訊息，讀取完畢或違反協定即斷開 Client 連線
客製化通訊協定	開發者可實作不同的 Tunnel 類別（如 Waterboard Tunnel），並覆寫工廠方法或實作 TunnelFactory 來使用

🔑 重要觀念總結

觀念	說明
工廠方法（Factory Method）	一個抽象方法，延遲決定實際要創建的產品類別，通過繼承覆寫來指定實體類型。是樣板方法的一種應用。
開放封閉原則（OCP）	程式碼對擴充開放，對修改封閉，工廠方法允許新增 Tunnel 類別而無需改動 Chat Server 類別。
一次性物件限制	Tunnel 物件用完即棄，必須為每個 Client 動態建立新物件，無法事先依賴注入固定實例。
組合取代繼承	為避免多變行為引起的類別爆炸，將變動行為委派給工廠介面，用依賴注入的方式組合。
組合爆炸（Combinatorial Explosion）	多個變動行為透過繼承組合時，可能導致類別數量呈指數成長，須透過委派解決。

✨ 工廠方法模式的設計條件與適用情境

條件	解釋
依賴的介面在使用上有生命週期限制或一次性使用限制	例如 Tunnel 介面只能使用一次，需要在適當時機創建新的實例。

需為每個需求或對象創建不同的實作物件	每個 Client 需要不同 Tunnel，或每次連線都需新物件。
未來會有多種實作類別，且必須保持擴充性，避免修改既有程式碼	希望開發者能在不改動 Chat Server 類別的情況下新增不同協定的 Tunnel。
不能單純使用依賴注入取代工廠方法	由於物件的生命週期限制及動態創建需求，無法直接將實例注入到 Context 中。

Threadwave.ai

🧠 工廠方法與樣板方法的關係

- 工廠方法是樣板方法的一種特殊應用，專注在「創建行為」的延遲決定。
- 樣板方法將整體流程中「變動」與「不變」部分區分開，工廠方法則聚焦於「變動的創建物件類型」。
- 兩者皆利用繼承與多型，讓子類別決定變動部分的實現。

💣 過度設計（Overdesign）警示

- 不要因為想「輕鬆替換實作類別」而濫用工廠方法。
- 若只是單純想替換依賴物件，使用 **依賴注入（Dependency Injection）** 並實現介面即可，無需工廠方法。
- 工廠方法真正解決的是「必須在執行時動態、重複創建且類型未知的物件」，且「必須保留擴充彈性」的設計難題。
- 濫用工廠方法會製造不必要的複雜度與技術債。

🛠️ 工廠方法的經典實作流程

1. **辨識變動與不變：**
   找出行為中會改變的部分（創建物件），與不會改變的部分（整體流程）。

Thetawave.ai

2. **將變動行為定義為抽象工廠方法：**
   在 Context 類別中宣告一個抽象方法（如 CreateTunnel），用來創建物件。
3. **子類別覆寫工廠方法：**
   讓子類別決定實際創建哪一種產品類別。
4. **呼叫工廠方法獲取具體物件：**
   Context 在適當時機呼叫工廠方法，取得具體產品，完成後續行為。
5. **（可選）使用組合與介面委派：**
   為避免類別爆炸，將工廠方法委派給工廠介面，並透過依賴注入組合。

💡 工廠方法的類別圖示意

類別	角色	方法與說明
Context (抽象類別)	定義範本流程及抽象工廠方法	StartServer()、abstract CreateTunnel(Client)
ConcreteContext	實作 Context，覆寫工廠方法指定產品類型	CreateTunnel(Client) 回傳具體 Tunnel 實例
Product (介面)	定義產品的共通介面	Message()、Disconnect()
ConcreteProduct	Product 的具體實作	實作通訊協定細節
Factory (介面)	（重構後）定義工廠介面	CreateTunnel(Client)
ConcreteFactory	實作 Factory，回傳具體產品	負責產出特定 Tunnel 實作

🔍 範例比喻說明

- **Tunnel 物件如「一次性免洗筷」**：用完即丟，不能重複使用，必須每次用餐（Client 連線）時提供新的免洗筷（Tunnel）。
- **工廠方法如「餐廳點餐菜單」**：餐廳（Chat Server）不必知道廚師（子類別）如何做菜，餐廳只負責點菜（呼叫工廠方法），由廚師決定菜色（具體 Tunnel 實作）。

- **組合取代繼承如「菜單外包給專業廚師」**：餐廳不需自己研發菜單，只需委派菜單給專業廚師（工廠介面），以避免菜色爆炸導致餐廳難以管理。

Mindwave.ai

⚖️ 工廠方法的設計價值判斷

- 工廠方法有效解決因「物件生命週期限制」與「擴充彈性需求」所產生的困難。
- 透過工廠方法，系統能夠在不修改既有程式碼的情況下靈活新增產品類別。
- 配合組合取代繼承原則，可避免因多個變動點導致的類別爆炸問題，提高系統可維護性。
- 工廠方法的使用需建立在明確且必要的需求基礎上，非必要時避免過度設計。